



Classe LMD S5 Informatique
Année Universitaire : 2020 / 2021
Mr Benaissa Mohamed

Date 08/03/2021

Solution Examen N°1 : Module système d'exploitation

Questions de cours

Q1) Répondez par VRAI ou FAUX

- a) Tant qu'un processus est bloqué dans la file d'attente d'un sémaphore, il est engagé dans l'attente active.

Faux (il est engagé dans l'attente passive) //1 pt

- b) L'exclusion mutuelle peut être réalisée avec un sémaphore général dont la valeur initiale est supérieure à 1.

Faux (la valeur initiale de sémaphore = 1) // 1pt

Q2) Quelle est la différence entre un blocage et interblocage dans l'accès concurrence a des ressources critiques ?

Interblocage : plusieurs processus bouclent en parallèles et la section critique est libre.

Blocage : il existe un processus qui n'est pas intéressé par la section critique est empêche les autres processus de ne pas entrés dans la section critique. //1 pt

Q3) Expliquez chacun des termes suivants en précisant la différence principale entre les deux termes. - "sémaphore binaire" et " sémaphore général"

Déférence entre sémaphore binaire et sémaphore général

Ils sont les deux utilisés pour régler l'accès aux ressources partagées.

1) Un sémaphore binaire:

- a une valeur 0 ou 1 / (exclusion mutuelle)

- convient pour seule instance d'une ressource// 0.5 pt

2) Un sémaphore général :

- a une valeur entière de 0 à n.

- convient pour plusieurs ressources de même type (identiques). //0.5 pt

Q4) quel est le problème se produit dans la gestion de synchronisation par le moniteur lorsque le signal est exécuté avant wait ? c'est a dire le signal est perdu avant le blocage d'un processus par wait .

Il y a une situation de blocage de votre programme en cours d'exécution //1 pt

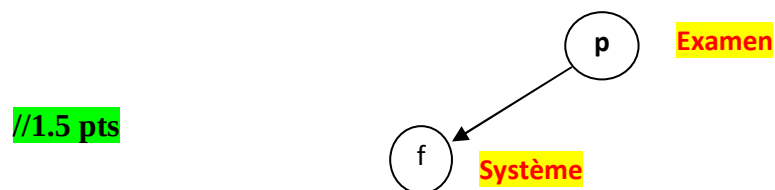
Exercice 1

Vous avez le programme suivant :

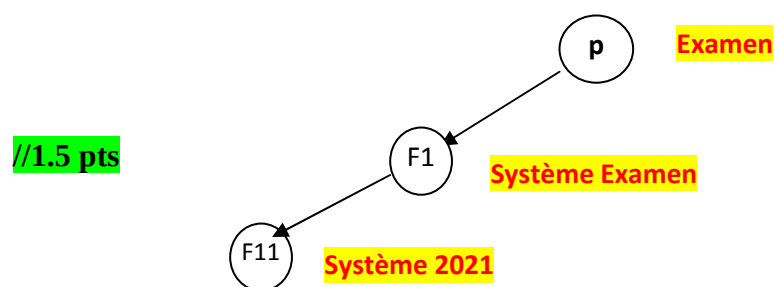
```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>

int main() {
    int i=0;
    while (i < 2) {
        if (fork())
        {
            printf( "examen\n " ) ;
            i++ ;exit(0);}
        else {printf( "systeme\n" ) ; i++; exit(0); }
    }
    printf("2021\n");
}
```

Q1 : Donnez l'arbre des descendance de processus, et l'affichage de chacun d'entre eux.

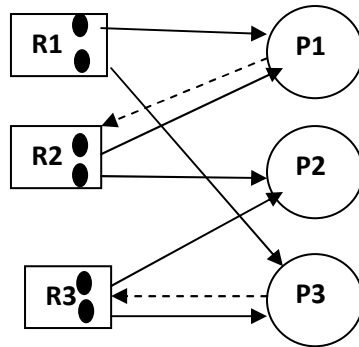


Q2 : si on supprime exit(0) souligné, quelle est la nouvelle arborescence des processus ?



Exercice 2

Nous considérons le graphe d'allocation de ressources suivantes :



- 1) Donnez la matrice allocation, la matrice requête, le vecteur total et le vecteur disponible

Vecteur total = (R1=2, R2=2, R3=2)

Vecteur disponible (R1=0, R2=0, R3=0) //1 pt

	R1	R2	R3
P1	1	1	0
P2	0	1	1
P3	1	0	1

Matrice allocation

	R1	R2	R3
P1	0	1	0
P2	0	0	0
P3	0	0	1

Matrice requête

- 2) Vérifiez si le système se trouve dans un état sain (stable) pas de situation d'interblocage.

Avec vérification : etat sain : p2 -> p1 -> p3 //1 pt

- 3) Nous Supposons maintenant que P2 demande la ressource R1. Ce graphique d'allocation de ressources contient-il une situation d'interblocage?

Avec vérification : il y a une situation d'interblocage entre p1, p2 et p3 //1 pt

- 4) Nous Supposons dans le graphe d'allocation de ressources au point 3), qu'il y a trois instances (exemplaires) de la ressource R2. Ce graphique d'allocation de ressources contient-il une situation d'interblocage?

Avec vérification : etat sain : p1 -> p2 -> p3 //1 pt

- 5) Nous ajoutons au graphe d'allocation de ressources d'origine un processus supplémentaire P4 qui demande une instance de R1. Le graphique d'allocation contient-il une situation d'interblocage?

Avec vérification : etat sain : p2 -> p1 -> p3 -> p4 //1 pt

Exercice 3 / TEST TP SYSTEME /

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
```

```
void *T1(void *arg) {
printf("T");
printf(" E/n");
pthread_exit(NULL);
}
```

```
void *T2(void *arg) {
printf("S");
printf("M");
pthread_exit(NULL);
}
```

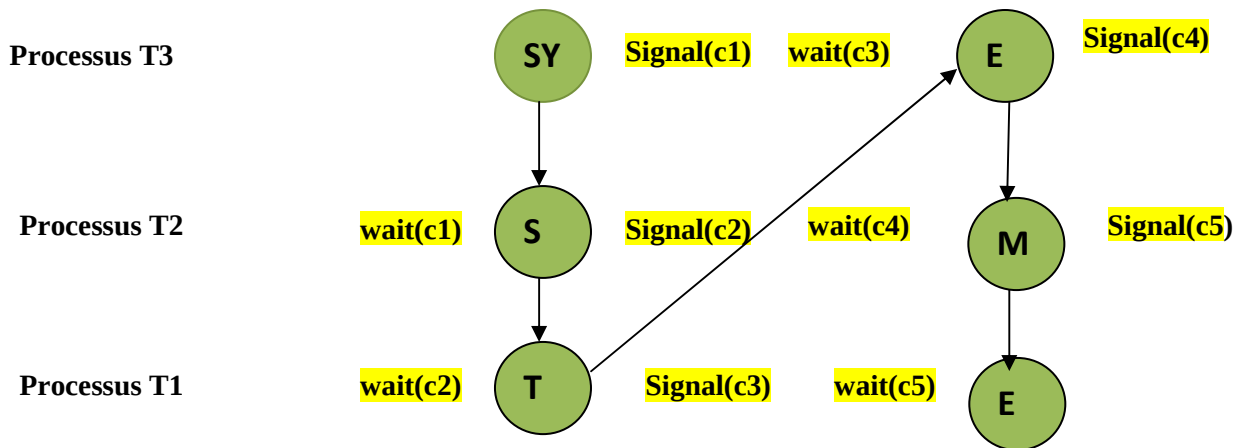
```
void *T3(void *arg) {
printf("SY");
printf(" E");
pthread_exit(NULL);
}
```

```
int main(){
pthread_t tid1, tid2;
pthread_create(&tid1,NULL,T1, NULL);
pthread_create(&tid2,NULL,T2, NULL);
pthread_create(&tid3,NULL,T3, NULL);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
pthread_join(tid3,NULL);
exit(0);
}
```

Q1: complétez et synchronisez les 3 tache T1, T2 et T3 en utilisons **les moniteurs** pour afficher le mot suivant : **SYSTEME**

Solution Exercice 3 / TEST TP SYSTEME /

//4 pts



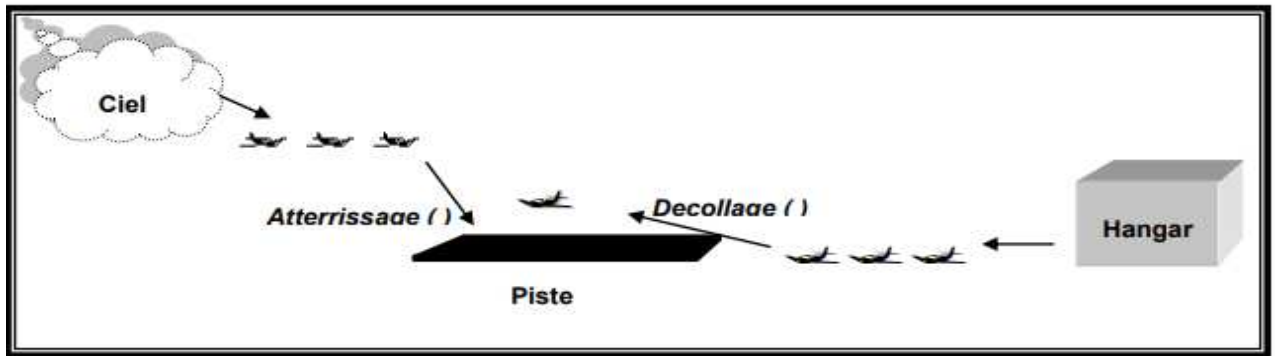
Exercice 4

Le but de cet exercice est la gestion du trafic aérien. On ne dispose que d'une seule piste (section critique) à la fois d'atterrissage et de décollage. En plus, cette piste ne peut accepter qu'un seul avion quelque soit la manœuvre (atterrissage ou décollage). Pour cela, on dispose de deux files d'attente :

- en l'air de taille N pour les avions souhaitant atterrir
- et au sol de taille M pour les avions souhaitant décoller.

La gestion de ce trafic d'avions nécessite, alors, quatre fonctions :

- ✓ une fonction `SortirAvions ( )` qui fait sortir les avions du hangar (dépôt) et les place dans la file d'attente de décollage,
- ✓ une fonction `Decollage ( )` qui prend un avion cloué en sol dans la file d'attente de décollage et le fait décoller en utilisant la piste,
- ✓ une fonction `AmenerAvions ( )` qui fait entrer, dans la file d'attente d'atterrissage, des avions en vol,
- ✓ et une fonction `Atterrissage ( )` qui prend un avion de la file d'attente d'atterrissage et le fait atterrir en utilisant la piste.



1. Quelle est l'utilité de chaque sémaphore ? **c'est la synchronisation** //0.5 pt
2. indiquez la relation qui existe entre la fonction atterrissage et la fonction decollage.
C'est la relation d'exclusion mutuelle //0.5 pt
3. Donner une solution au problème en complétant le pseudo code suivant en utilisons les sémaphores :

```
#define N 5
#define M 5
semaphore Decollage_vide, Decollage_plein, Atterrissage_vide, Atterrissage_plein,
Piste;
Sem_Init (Decollage_vide, M);
Sem_Init (Decollage_plein, 0);
Sem_Init (Atterrissage_vide, N);
Sem_Init (Atterrissage_plein, 0);
Sem_Init (Piste, 1);
```

```
void SortirAvions ( )
```

```
{
```

```
P(Decollage_vide)
```

```
Ajouter_un_avion_dans_la_zone_attente_decollage();
```

```
V(Decollage_plein) //0.75 pt
```

```
}
```

```
void Decollage ( )
```

```
{
```

```
P(Decollage_plein)
```

```
P(piste)
```

```
Faire_decoller_un_avion ( );
```

```
V(piste)
```

```
V(Decollage_vide) //0.75 pt
```

```
}
```

```
void AmenerAvions ( )
```

```
{  
P(Atterrissage_vide)  
Ajouter_un_avion_dans_la_zone_attente_atterrissage();  
V(Atterrissage_plein) //0.75 pt  
}
```

```
void Atterrissage ( )  
{  
P(Atterrissage_plein)  
P(piste)  
Faire_Atterrir_un_avion ( );  
V(piste)  
V(Atterrissage_vide) //0.75 pt  
}
```